NVLLM: A 3D NAND-Centric Architecture Enabling Edge on-Device LLM Inference

Mingbo Hao, Changwei Yan, Haoyu Cui, Zhihao Yan, Yizhi Ding, Zhangrui Qian, Weiwei Shan*
School of Integrated Circuits, Southeast University, Nanjing, China

Abstract

The rapid growth of LLMs demands high-throughput, memory-capacity-intensive inference on resource-constrained edge devices, where single-batch decoding remains fundamentally memory-bound. Existing out-of-core GPU-based and SSD-like accelerators are limited by DRAM-bound weight movement and inefficient storage access granularity. We present NVLLM, a 3D NAND-centric inference architecture that offloads feed-forward network (FFN) computation into the Flash while executing attention on lightweight CMOS logic with external DRAM. Through wafer-to-wafer stacking, NVLLM tightly integrates multi-plane 3D NAND with compute pipelines, error correction code (ECC) units, and buffers, enabling page-level FFN weight access without DRAM traversal. All GEMM/GEMV operations are decomposed into dot-product primitives executed by out-of-order PE lanes, operating directly on raw NAND reads with integrated ECC. Attention weights remain in DRAM, and a KV-cache-aware scheduler sustains throughput as the context length grows. Evaluated on OPT and LLaMA models with up to 30B parameters, NVLLM achieves a 16.7×–37.9× speedup over A800-based out-of-core inference and up to 4.7× speedup over SSD-like designs, with only 2.7% CMOS area overhead.

Keywords

3D NAND Flash, In-Storage Computing, Near-Data Processing, Large Language Models, On-Device AI, Error-Resilient Computing

1 Introduction

Deploying large language models (LLMs) on edge devices is increasingly appealing for privacy, latency, and offline availability [38, 39, 44]. However, achieving accurate and efficient on-device inference remains challenging due to strict constraints on memory capacity, bandwidth, and power. Unlike cloud servers, edge devices cannot exploit large-batch parallelism; interactive requests arrive irregularly, and decoding predominantly proceeds in a single-token, single-batch manner [39]. As shown in Fig. 1(a), cloud workloads naturally form high arithmetic intensity [11, 41], whereas edge workloads do not.

Emails: mingbohao@seu.edu.cn (Mingbo Hao); wwshan@seu.edu.cn (Weiwei Shan, corresponding author).

Mingbo Hao is a Ph.D. student at Southeast University, pioneering 3D NAND-centric computing architectures for large language model inference acceleration. This document is an author-prepared preprint version of the manuscript, with publication-specific formatting elements removed for improved readability. The work presents a system-level exploration of 3D NAND-centric LLM inference acceleration under heterogeneous memory hierarchies, with NVLLM serving as an initial system instantiation of this research direction. This version incorporates minor refinements from the conference version while preserving the core design, evaluation methodology, and conclusions. It is released for academic dissemination and discussion.

To characterize this behavior, Fig. 1(b) analyzes 6.1M conversation turns from ShareGPT [32], UltraChat [5], and OASST1 [29]. User prompts are overwhelmingly short, while model responses exhibit long tails. This "short-prompt, long-generation" pattern implies that edge inference is dominated by the decode phase, where each token requires a full forward pass without batch-level parallelism.

Fig. 2 shows a roofline analysis across representative hardware platforms [2, 13, 27, 28]. Under single-batch, long-generation workloads, performance is heavily memory-bound during both prefill and decode. For instance, maintaining a 3 tokens/s user-perceived rate [10, 15] for a 70B model requires only 0.42 TOPS of theoretical compute—comparable to the per-token rate of an RTX 4090 running 6-bit LLaMA3.3-70B [9]—yet its peak compute is over 3000× higher. A multi-turn ShareGPT trace further confirms that decode iterations lie far below hardware compute ceilings but remain bottlenecked by weight loading.

Since batch-level parallelism is fundamentally limited on the edge, datacenter-oriented optimizations [30] lose effectiveness. Edge-side approaches such as speculative decoding [23], lookahead decoding [8], early exit [34], and quantization/sparsity [36] remain limited by DRAM bandwidth. Increasing DRAM bandwidth (e.g., HBM [16]) raises power and cost; insufficient DRAM capacity forces repeated parameter streaming [33], further increasing latency and energy. Thus, scaling DRAM alone is not a viable solution for single-batch inference.

3D NAND Flash provides an alternative path. Single-token decoding offers substantial compute slack, enabling near-data computation to amortize data movement costs. Wafer-to-wafer (W2W) 3D integration [17, 40] stacks CMOS beneath or above the NAND array, allowing lightweight compute without sacrificing density. Modern Flash products already adopt such structures [14, 35], combining high density and low cost per bit with the ability to access multi-megabyte feed-forward network (FFN) weights in situ.

However, several challenges arise. Raw NAND reads contain bit errors requiring error correction code (ECC), yet in-Flash computation precedes conventional correction. Flash writes are slow (200–800 μs program, 3–5 ms erase) and endurance-limited [7], constraining write-heavy schemes. Meanwhile, KV-cache growth increases attention cost and can degrade decode throughput if unmanaged.

We propose NVLLM, a **3D NAND-centric** architecture addressing these challenges. FFN weights are stored in Flash, while attention weights reside in DRAM. GEMM/GEMV operations are decomposed into dot-product primitives executed by an out-of-order in-Flash pipeline. By decoupling error detection from correction, the pipeline operates directly on raw NAND reads, sustaining throughput while tolerating bit errors. Keeping attention weights

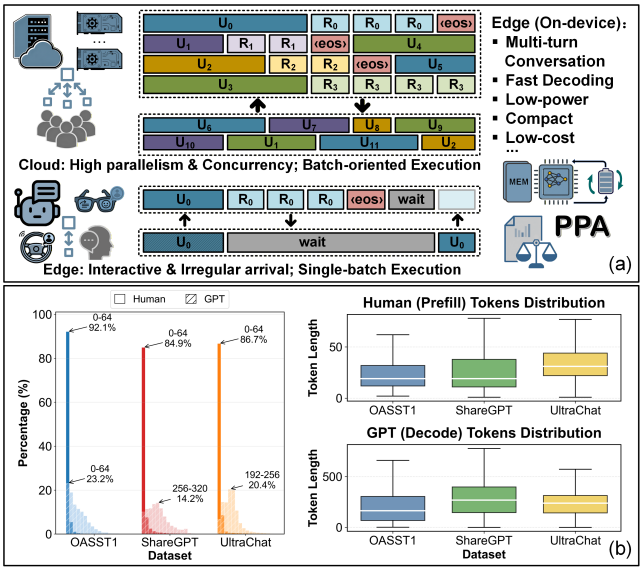

Figure 1: (a) Comparison of edge and cloud LLM inference devices and their execution characteristics. (b) Analysis of the token-distribution patterns in human–LLM interactive conversations.

in DRAM reduces bandwidth pressure and preserves the random-access demands of the attention path. A KV-cache–aware scheduler monitors cache growth and dynamically offloads portions of Q/K/V/O projections to the in-Flash pipeline, maintaining stable per-token throughput under long contexts.

Building on these design principles, NVLLM shows that combining in-Flash FFN execution, DRAM-resident attention weights, and KV-cache–aware scheduling effectively improves token throughput, reduces off-chip memory traffic, and scales to larger models under edge constraints. Evaluated on quantized OPT and LLaMA models up to 30B parameters, NVLLM achieves 16.7×–37.9× higher token throughput than an out-of-core GPU under the same weight-offloading strategy, and up to 4.7× higher throughput over SSD-like in-Flash designs, while incurring only 2.7% CMOS area overhead. The key contributions of this work are as follows:

- We propose NVLLM, a 3D NAND-centric architecture for edge-side LLM inference. By co-designing compute and storage through wafer-level 3D integration, NVLLM performs FFN computation inside Flash and substantially reduces off-chip memory traffic.
- We design an in-Flash dot-product engine that sustains high throughput under raw NAND read errors. Decoupling error detection from correction enables out-of-order, error-tolerant vector execution and unlocks an efficient area-power design space.
- We show that attention and FFN exhibit weak interdependence during inference and exploit this property with a decoupled execution strategy. A KV-cache-aware scheduler further preserves token throughput by adaptively balancing attention workloads in long-context scenarios.

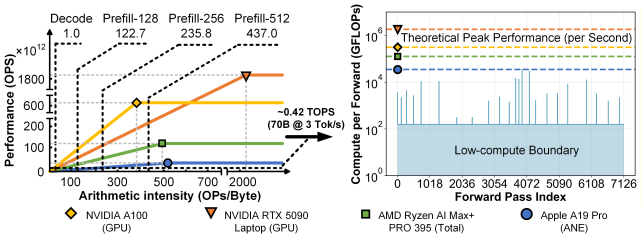

Figure 2: Under the single-batch inference constraint, the hardware remains heavily memory-bound for the majority of the workload.

2 Background and Motivation

2.1 Large Language Models at the Edge

Most modern LLMs adopt multi-layer Transformer decoder architectures [43], where each decoder layer consists of an attention module and a FFN. Attention typically employs multi-head attention (MHA), computing queries (Q), keys (K), and values (V) via linear projections, followed by scaled dot-product, causal masking, softmax, and weighted aggregation. FFNs expand the hidden dimension (e.g., 3.5× in LLaMA3-8B) and project it back, accounting for roughly 70% of total parameters—around 5.3 GiB under 8-bit quantization. Both attention and FFN maintain input dimensions through residual connections and layer/RMS normalization.

Edge deployments face fundamentally different workloads than cloud servers [41]. Interactive single-batch prompts dominate, leaving little room for parallelism and resulting in underutilized compute. Workload analysis (Fig. 3(a)) confirms that FFNs dominate model parameters, while single-batch decoding dominates runtime compute, making external memory access (EMA) the main energy and throughput bottleneck. Multi-turn interactions increase KV-cache size, shifting attention compute toward aggregation over Q/K/V/O projections (Fig. 3(b)). These characteristics emphasize the memory- and bandwidth-bound nature of edge LLM inference and highlight the need for architectures tailored to these constraints.

2.2 3D NAND Flash for Edge LLMs

Traditional attention-focused accelerators [24, 37, 45] typically store all weights in DRAM, incurring high energy and bandwidth costs (56–69% of total system power) and up to 1.5× extra computation. Scaling DRAM alone is costly and often insufficient for single-batch edge inference.

3D NAND Flash provides an opportunity to offload FFN computation near memory. Single-token decoding offers ample compute slack, while page-level access retrieves multi-gigabyte weights in situ. W2W 3D integration [12, 17] allows CMOS logic to reside beneath the NAND array, enabling high-density and area-efficient computation. Challenges include non-negligible raw bit error rate (RBER) in NAND Flash memory, limited write endurance [7], and restricted CMOS area for logic [14, 26], as illustrated in Fig. 4.

Decoupling FFN and attention computation enables efficient in-Flash FFN execution while retaining attention in DRAM, where irregular data dependencies and growing KV caches are better supported. FFN pipelines in Flash can exploit regular, weight-resident

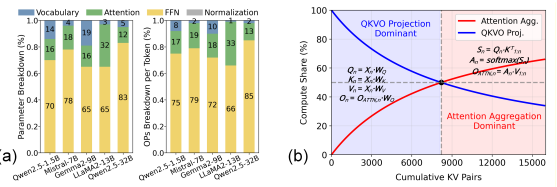

Figure 3: (a) Breakdown of model parameters and per-token OPs (single batch); Normalization is shown in the legend even if negligible. (b) Attention computation growth with increasing KV-cache size in long-context inference.

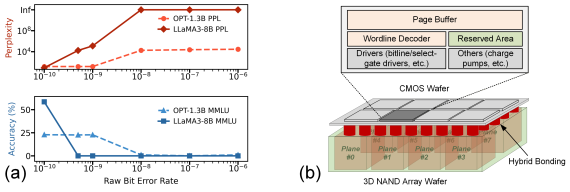

Figure 4: (a) NAND read-induced RBER increases perplexity and lowers accuracy. (b) CMOS area constraints in Flash logic require lightweight, error-tolerant computation pipelines.

access patterns with minimal on-chip buffering, while attention benefits from DRAM bandwidth and algorithmic optimizations. This division improves throughput, reduces off-chip traffic, and scales to longer contexts, motivating a NAND-centric edge architecture for LLM inference.

3 NVLLM Architecture

Fig. 5 illustrates the overall architecture of NVLLM, a wafer-to-wafer co-designed system that executes FFN layers directly within 3D NAND Flash, while performing attention and KV-cache operations on the DRAM-side NPU. This division of labor matches the intrinsic characteristics of LLM workloads: FFN layers hold the vast majority of parameters with highly regular, weight-stationary access patterns and minimal activation footprint, whereas attention computation is dominated by cache-resident Q/K/V/O projections and KV-dependent aggregation whose irregular data reuse is best served by DRAM bandwidth.

By combining hybrid-bonded CMOS logic with Flash planes, NVLLM eliminates DRAM round-trips for FFN weights, reduces EMA energy, and frees bandwidth for attention. The architecture is organized around three principles: (i) structured decoupling of attention and FFN compute, (ii) plane–lane co-alignment for weight streaming efficiency, and (iii) error-resilient, out-of-order FFN execution that matches NAND read behavior.

3.1 Clustered Plane-Lane Co-Design

3D NAND exposes data at page granularity (4–16 KiB), but page-level bandwidth from a single plane is insufficient to saturate compute pipelines. NVLLM therefore groups planes into *clusters*, each comprising $k \times k$ planes sharing peripheral circuitry and synchronously returning page-buffer (PB) rows. This arrangement offers two co-design advantages. First, a cluster aggregates the per-plane read bandwidth and amortizes PB/peripheral-circuit overhead, which is particularly beneficial in W2W-bonded 3D NAND, where the CMOS periphery beneath each array footprint is area-constrained. Second, and more importantly, the cluster width is chosen to *match the data consumption width of PE lanes*. Each PE lane processes a segment of d weights per cycle, and a cluster produces d bytes per activation step on average. This alignment eliminates wide multiplexers, avoids cross-lane reshaping, and enables direct streaming from cluster-level FIFOs (CFs) into lane-level segment buffers with a single narrow handshake. Without such co-alignment, implementing a lane-scaled dot-product pipeline within the NAND CMOS wafer would incur prohibitive buffering and routing overhead.

Each cluster thus forms a self-consistent weight delivery unit with three buffering layers: (i) plane-level PBs, (ii) a shallow cluster FIFO that absorbs WL-level read latency, and (iii) per-lane segment buffers that gate data into the compute pipeline. These layers together hide WL RC delay, sense-amplifier variation, and multi-V_{read} cycles for MLC/TLC cells, enabling deterministic, stall-free weight delivery to FFN compute.

3.2 Error-Resilient Dot Product Algorithm

At the heart of in-Flash FFN execution is the error-resilient dot-product engine (ERDPE), which performs mixed-precision (BF16 or INT8) dot products directly on raw NAND reads. To address bit errors inherent to in-Flash computing, NVLLM decouples error detection from correction and introduces a lightweight out-of-order (OoO) dot-product pipeline that implements an error-corrected dot-product (ECDP) in an out-of-order manner (OoO-ECDP).

To realize this execution model in hardware, NVLLM adopts algorithm 1 that decomposes each weight column into multiple independent segments and schedules them based on error-check readiness. This algorithm provides a hardware-friendly way to overlap error detection, multi-cycle correction, and MAC execution without requiring the correction units to finish beforehand.

3.3 ERDPE Microarchitecture

ERDPE consists of multiple OoO-ECDPs, each containing PE lanes that execute segment-level dot products 5(b–c). For each weight segment, a lane first performs an inline error check. If no error is detected, the segment proceeds directly to MAC accumulation. Otherwise, the segment is forwarded to a shared multi-cycle corrector hub, while the lane continues processing other ready segments. Corrected segments returned from the hub update the partial sums tracked by the scoreboard.

Each OoO-ECDP implements the OoO dot-product algorithm in hardware and contains: (i) a segment buffer providing $2\times d$-wide weight fragments, (ii) a bit-error checker, (iii) a mixed-precision MAC unit, and (iv) a commit buffer with scoreboard interfaces that track segment readiness and pending corrections.

Segments are scheduled by readiness rather than strict order, making each lane resemble a narrow out-of-order pipeline. The scoreboard tracks whether a segment has passed error detection (valid bit = 0), is waiting for correction (valid bit = 1), or has been committed (no entry). This allows the lane to continue executing other ready segments even while others undergo multi-cycle correction. Final accumulation follows the in-order completion of weight

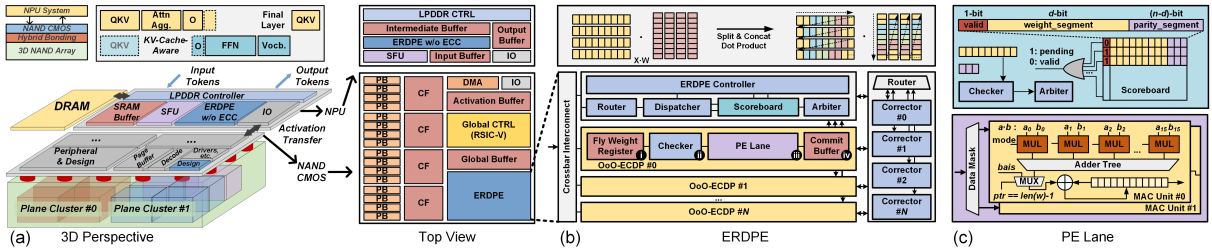

Figure 5: Proposed NVLLM architecture.

Algorithm 1 Out-of-Order Dot Product for Error-Resilient

Input: Weight vector $\mathbf{w} \in \mathbb{R}^{h\times 1}$, Activation vector $\mathbf{a} \in \mathbb{R}^{1\times h}$, Segment factor d, Error-correcting code $L(n,d)$, Parity matrix $\mathbf{p} \in \mathbb{R}^{(n-d)\times(h/d)}$,

Output: Final dot product result s

Initialize accumulator $s \leftarrow 0$, scoreboard $B \leftarrow \emptyset$
Weight vector pointer $ptr \leftarrow 0$
while $ptr <$ len($\mathbf{w}$) **do**
  $\mathbf{w}_d \leftarrow \mathbf{w}[ptr : ptr + d - 1]$ ▷ Current weight segment
  $\mathbf{p}_{n-d} \leftarrow \mathbf{p}[0 : n - d, ptr]$ ▷ Parity segment
  $\mathbf{v} \leftarrow$ Concat$(\mathbf{w}_d, \mathbf{p}_{n-d})$
  if Checker$(\mathbf{v}, L(n,d))$ **then**
    $s \leftarrow s + \mathbf{w}_d \cdot \mathbf{a}[ptr : ptr + d - 1]$
  else ▷ Non-blocking
    $B \leftarrow B \cup \{ptr\}$
    $\mathbf{w}' \leftarrow$ Corrector$(\mathbf{w}_d, \mathbf{p}_{n-d})$
    $\mathbf{w}[ptr : ptr + d - 1] \leftarrow \mathbf{w}'$
  end if
  $ptr \leftarrow ptr + d$
end while
if $B \neq \emptyset$ **then**
  for $idx \in B$ **do**
    $\mathbf{w}_d \leftarrow \mathbf{w}[idx : idx + d - 1]$
    $s \leftarrow s + \mathbf{w}_d \cdot \mathbf{a}[idx : idx + d - 1]$
  end for
end if
return s

columns, ensuring numerical correctness and matching the fixed page-level read address accumulation mode of the plane side.

A fly-weight register guarantees that each lane consumes exactly one segment per cycle without reshaping. In this design, the double-buffer structure also ensures that when a flying segment encounters an error, the MAC unit can immediately consume another weight segment. This is implemented via a bypass path between the fly-weight register and the PE lane. Given the extremely low probability of consecutive segment errors, it is reasonable to assume that another segment can be executed without delay. To avoid any potential loss of accuracy, once an OoO-ECDP enters this state, the faulty buffer is temporarily masked until the next cycle, when the data and corresponding check bit are transferred by the arbiter to the scoreboard. The segment is then automatically corrected. If the corrected weight segment matches the original, the scoreboard removes the entry directly.

This architecture allows the pipeline to sustain a full-cycle MAC rate even under non-zero RBER, while amortizing the correction logic across lanes. The datapath includes fused multiply-accumulate units supporting BF16 and INT8 execution, with optional widening for high-dynamic-range Transformer layers.

3.4 Structured Prefetch and Pipeline Scheduling

Because NAND reads are deterministic at page granularity, prefetching can be tightly coordinated with lane-level compute. NVLLM issues prefetches on a per-cluster, per-layer schedule, filling cluster FIFOs ahead of ERDPE consumption without generating additional random reads. The scheduler integrates with the OoO mechanism: segments with earlier deadlines are prefetched first, while segments experiencing correction stalls do not throttle the pipeline. This structure contrasts with DRAM-oriented prefetchers that must predict irregular access patterns; here, FFN weights are laid out contiguously across plane clusters, enabling deterministic, deadlock-free scheduling.

3.5 End-to-End Dataflow

As a 3D NAND-centric architecture, NVLLM coordinates data flow across the NAND array, NAND CMOS, NPU, and DRAM. To minimize energy-intensive transfers, recurrent weight accesses remain within either the 3D NAND or the NPU-DRAM subsystem; notably, Q/K/V/O weights are copied once into DRAM at initialization, eliminating all subsequent Flash-to-DRAM movement.

During prefill, the global scheduler distributes Q/K/V/O computation between NAND CMOS and the NPU based on their effective compute capabilities. The ERDPE performs in-Flash linear projections, after which the NPU fuses the Q, K, and V matrices, executes attention, and stores K/V in DRAM. Multi-head output projection follows an identical compute split, while FFN computation and the final output projection are both executed on the NAND side. The resulting logits are then transferred to the NPU for post-processing.

Decode follows a similar pipeline but processes a single token per step. At the early stage of decoding, attention is executed entirely on the NPU, while FFN computation is offloaded to the NAND side. As the KV cache grows, NPU-side attention latency becomes non-stationary, leading to imbalance in the shared Q/K/V/O path. NVLLM addresses this via a KV-cache-aware dynamic scheduler that adaptively adjusts the NAND-NPU compute split in real time using a lightweight latency estimator and a bitmap-based dispatcher. Algorithm 2 illustrates a concrete instantiation of the proposed adaptive scheduling mechanism.

Algorithm 2 KV-Cache-Aware Scheduling

Input: Cycle increment ΔC (NPU cycles), NPU latency per column C_{NPU}, weight column size u, page buffer per plane cluster P, cluster-level bitmap $B^{(n)} \in \{0,1\}^{1\times H}$
Output: Updated bitmap $B^{(n+1)}$
Executed at the end of each forward propagation

$C_{\text{th}} \leftarrow \lfloor P/u \rfloor \cdot C_{\text{NPU}}$
if $\Delta C \le C_{\text{th}}$ **then**
  return $B^{(n)}$
end if
$k \leftarrow \lceil \Delta C / C_{\text{th}} \rceil$
$B^{(n+1)} \leftarrow B^{(n)}$, $cnt \leftarrow 0$, $i \leftarrow H-1$
while $cnt < k$ **and** $i \ge 0$ **do**
  if $B^{(n)}[i] = 1$ **then**
    $B^{(n+1)}[i] \leftarrow 0$
    $cnt \leftarrow cnt + 1$
  end if
  $i \leftarrow i - 1$
end while
return $B^{(n+1)}$

4 Evaluation

4.1 Methodology

Hardware Configuration. We model the 3D NAND Flash array at the plane level using the 3D-FPIM framework [19], which captures key technology parameters such as cell and wordline geometries, parasitic capacitances, and timing characteristics validated against commercial products. The W2W stacking in NVLLM is reflected by adapting the layout hierarchy in area estimation, constrained by teardown analyses of consumer-grade NAND products [14]. LPDDR5X DRAM serves as shared memory for Q/K/V/O weights and the KV cache, with timing and energy modeled using Ramulator2 [25] and DRAMPower [4]. A cycle-accurate C++ simulator integrates the 3D-FPIM NAND and DRAM behavioral models to evaluate system-level performance under diverse LLM workloads.

The NAND subsystem includes 32 planes organized into 2×2 clusters, each with a 16 KiB page buffer, yielding a total capacity of 128 GiB. Each plane occupies 3.07 mm^2 and exhibits a read latency of 5.12 μs. We also evaluate scaled variants NVLLM-12C and NVLLM-16C to study architectural scalability.

Benchmark Models. We adopt the OPT series—OPT-1.3B, OPT-2.7B, OPT-6.7B, OPT-13B, and OPT-30B—as representative benchmarks covering a broad parameter range. To demonstrate NVLLM's generality, we also evaluate LLaMA-family models. All models are quantized to INT8 for on-device inference. Random bit flips are injected into model weights to emulate realistic NAND RBER patterns, assessing robustness under non-ideal storage.

Baselines and Metrics. We compare NVLLM against conventional GPU-based inference and SSD-like in-Flash computation designs. GPU-centric baselines include GPU-DRAM and GPU-SSD, using an NVIDIA A800 GPU with DRAM or NAND Flash SSDs for model weights, implemented via FlexGen [33]. SSD-like baselines comprise Cambricon-LLM [42] and AiF [20], which embed compute logic within Flash chips and coordinate multiple chips via SSD-style channels. NVLLM-16C is configured with 16 clusters and 64 planes for fair comparison with these designs. Table 1 summarizes the baseline configurations.

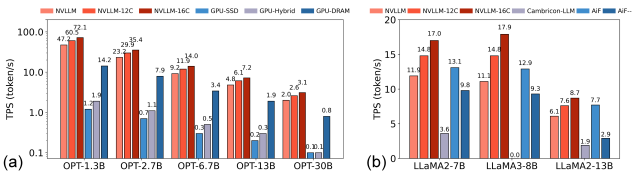

Figure 6: Throughput comparison result.

Performance metrics include tokens-per-second (TPS) for throughput, seconds per inference (s/inf) for end-to-end latency, and J/token for energy efficiency.

4.2 Area and Power Estimation

The NPU and NAND CMOS components of NVLLM were implemented in Verilog HDL and synthesized using Synopsys Design Compiler with TSMC 28 nm technology. Table 2 summarizes area and power breakdowns, reporting compute units and ECC logic separately. The ECC module achieves 2.0× area and 1.3× power reduction compared to a monolithic ECC design.

The NAND CMOS subsystem integrates SRAM blocks, including a 512 KiB cache FIFO (reusing internal Flash cache resources), a 72 KiB global buffer, and a 16 KiB activation buffer. A lightweight RISC-V CPU coordinates register initialization and runtime operations. Overall, the NPU occupies 0.46 mm^2, while the additional in-Flash logic occupies 2.69 mm^2—accounting for only 2.7% of the total NAND CMOS area.

4.3 Throughput

Comparison with GPU-centric Baselines. Although the NVIDIA A800 GPU delivers up to 624 TOPS of INT8 compute, generating one token of OPT-30B requires only 59.9 GOPs, leaving a substantial compute-utilization gap. Fig. 6(a) compares NVLLM, NVLLM-12C, and NVLLM-16C against three GPU-centric baselines. NVLLM operates its NAND CMOS at 350 MHz and the NPU at 500 MHz, delivering a total of 307–486 GOPS. All decoding tests are conducted under a 64-token context. Compared to GPU-SSD, NVLLM achieves 22.4×–37.9× speedup, leveraging multi-plane parallelism and internal 3D NAND bandwidth up to 100 GB/s, far exceeding typical PCIe Gen4 ×4 bandwidth limits of 8 GB/s. GPU-DRAM offers higher bandwidth than SSD but still falls short of NVLLM, yielding $>$ 2.5× lower throughput. Smaller models benefit more due to lower compute contribution relative to bandwidth.

Comparison with SSD-like Architectures. Fig. 6(b) compares NVLLM-16C with Cambricon-LLM, AiF, and AiF--. For LLaMA2-7B, NVLLM-16C outperforms these designs by 4.7×, 1.3×, and 1.7×, respectively. Cambricon-LLM's limited throughput (3.6 tokens/s) is due to shared Flash resources serving both in-Flash compute and NPU weight fetches. AiFSSD achieves 13.1 tokens/s with 102.4 GB/s internal bandwidth; AiF--, with reduced ECC/read optimizations, drops to 9.8 tokens/s.

NVLLM's advantage stems from hybrid-bonding enabling plane-level parallelism and higher internal bandwidth, and decoupled attention/FFN computation in NAND CMOS, allowing fully standalone inference without host-side coordination.

Table 1: Baseline Hardware Configuration Details

Architecture/System	GPU-Centric			SSD-Like			3D NAND-Centric
Design Name	GPU-DRAM	GPU-SSD	GPU-Hybrid	Cambricon-LLM	AiF	AiF--	NVLLM-16C
Weight Placement	DRAM	NAND Flash	NAND Flash + DRAM	NAND Flash			NAND Flash + DRAM
Quantization	8bit						
Hardware Configuration	NVIDIA A800, 80GB GPU; Intel Xeon Gold 6348 CPU; 512GiB DRAM, 16×32GiB DDR4; 5.2TB NVMe SSD			8ch, 2chip/ch, 2die/chip, 2plane/die, 64 planes, 16KiB page	8ch, 2chip/ch, 4plane/chip, 64 planes, 16KiB page		16 clusters, 4 planes/cluster, 64 planes, 16KiB page

Table 2: Area and Power Overhead of Compute Core

Component	Module	Area (μm^2)	Power (mW)
NPU	SFU	8,618	2.73
	Dot-Product Unit	144,712	170.40
	SRAM	304,217	67.00
	Others	1,767	0.02
	NPU Total	459,314	240.15
NAND CMOS	RISC-V CPU	685,284	2.76
	Dot-Product Unit	289,424	340.80
	Detector (x8)	82,256	159.69
	Corrector (x8)	323,608	107.66
	SRAM	1,292,922	284.75
	Others	18,089	0.02
	NCW Total	2,691,583	895.68

Table 3: NVLLM Scaling Configuration

Configuration	NVLLM	NVLLM-12C	NVLLM-16C
NPU	4×OoO-ECDP(w/o ECC), 2×LPDDR5X, 6×16Gb		
OoO-ECDP	8	12	16
Plane Cluster	8	12	16
Plane	32	48	64

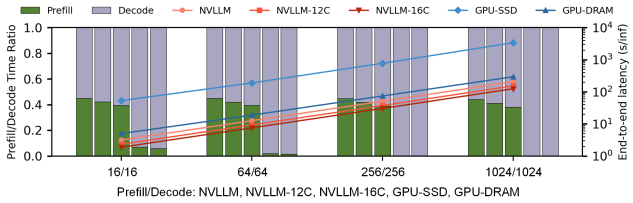

Figure 7: End to end latency comparison result.

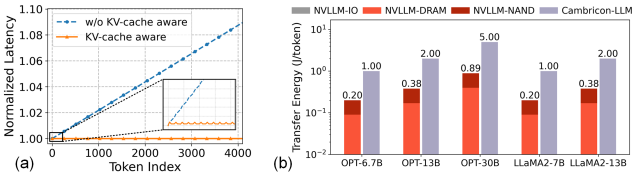

Figure 8: (a) The effect of KV-cache-aware scheduling. (b) Energy comparison result.

4.4 Latency Analysis

End-to-end latency of NVLLM is shown in Fig. 7, compared only to GPU-centric baselines due to missing prefill data for Cambricon-LLM and AiF. Prefill/decode token pairs range from 16/16 to 1024/1024. NVLLM distributes latency more evenly: prefill accounts for 44.1%–45.0% versus 0.1%–6.9% (GPU-SSD) and 0.15%–5.84% (GPU-DRAM).

This reflects fundamental differences: GPUs are compute-rich, making prefill fast and decode bandwidth-limited; NVLLM decomposes GEMM/GEMV into dot-product primitives, with 3D NAND + LPDDR5X supplying sufficient bandwidth for FFN and attention, keeping the prefill phase compute-bound. Increasing plane clusters with OoO-ECDP units (Table 3) further reduces prefill proportion. NVLLM-16C achieves 1.9 s, 7.5 s, 30.3 s, and 124.3 s per inference for 32, 128, 512, and 2048 tokens, up to 28.2× faster than GPU-SSD and 2.7× faster than GPU-DRAM.

Furthermore, the KV-cache-aware scheduling mechanism enhances NVLLM's latency benefits in extended decoding scenarios, which is validated by the ablation results shown in Fig. 8(a).

4.5 Energy Efficiency

NVLLM reduces data movement energy via decoupled attention/FFN design. Transfers occur along three paths: NAND array → NAND CMOS (NVLLM-NAND), and between NAND CMOS and the NPU (NVLLM-IO), and between the NPU and DRAM (NVLLM-DRAM). Most FFN weights remain within 3D NAND, and the decoupled architecture minimizes transfers between NAND CMOS and the NPU. Fig. 8(b) shows NVLLM's energy advantage over Cambricon-LLM. As model size increases, NVLLM's relative savings grow due to FFN-heavy workloads. NVLLM-IO is triggered only sparsely—primarily during layer transitions and the final output projection—and therefore contributes negligibly to the overall energy. Overall, NVLLM achieves 5.63× reduction in data movement energy compared to Cambricon-LLM.

5 Conclusion

We present NVLLM, a 3D NAND-centric architecture for on-device LLM inference, co-designing NAND Flash, CMOS logic, NPU, and DRAM to maximize internal bandwidth and minimize data movement. By decoupling attention and FFN computation, leveraging multi-plane parallelism, and adopting dynamic KV-cache-aware scheduling, NVLLM achieves up to 37.9× throughput improvement, 28.2× latency reduction, and 5.6× energy reduction over GPU- and SSD-based baselines. Our results demonstrate that carefully orchestrated near-data processing in 3D NAND enables efficient, scalable, and standalone LLM inference on resource-constrained devices.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant Nos. 92464204 and 92464302.

A Discussion: NVLLM Design Choices and Broader Implications

This appendix discusses the design rationale of NVLLM, its intended deployment scenarios, and its broader implications for memory-system-driven AI workloads.

A.1 Why a 3D NAND-Centric Architecture?

NVLLM is motivated by a simple observation: as models continue to grow, the memory system increasingly determines the overall efficiency of edge AI inference. In this context, many architectural variants are foreseeable. One could imagine more aggressive forms of in-storage execution, deeper coupling between Flash and programmable accelerators, or broader mappings of Transformer operators into the NAND subsystem. NVLLM does not rule out these possibilities. Instead, it deliberately chooses a narrower and more concrete first step.

The design starts from constraints that are already present in practical 3D NAND systems, including page-granular reads, limited CMOS logic budget, raw read errors, non-negligible access latency, existing storage interfaces, and the need to preserve conventional read/write behavior. Under these constraints, the key question is not how many operators can be theoretically mapped into Flash, but which part of the workload exposes a stable architectural bottleneck that current or near-term technology can realistically address.

More broadly, NVLLM follows a problem-grounded view of architecture research. Rather than treating architectural freedom itself as the starting point, the design starts from constraints that are visible in current and near-term technology, including device behavior, process limits, storage interfaces, reliability, and system integration cost. From this perspective, the role of architecture is to identify which emerging bottlenecks have become first-order system problems under these constraints, and what existing technologies must support in order to make such solutions practical. NVLLM is developed in this spirit: it treats 3D NAND Flash as a constrained but increasingly relevant execution substrate, and focuses on the part of LLM inference where model capacity, data movement, and edge-system resources already meet as an architectural problem.

NVLLM answers this question by targeting the weight-dominated and regular portion of LLM inference. This choice reflects a design philosophy rather than a lack of alternatives. The goal is to first solve the architectural problem that emerges when large model weights reside in high-capacity non-volatile storage but must be repeatedly moved through a conventional storage-memory-compute hierarchy. By focusing on this problem, NVLLM provides a concrete point in the design space for 3D NAND-centric inference.

A.2 Rationale Behind the FFN/Attention Partition

The FFN/attention partition in NVLLM follows directly from this philosophy. FFN layers account for a large fraction of model parameters and exhibit regular weight-streaming behavior. Their access pattern naturally aligns with the page-level organization and plane-level parallelism of 3D NAND Flash. Executing FFN computation close to Flash therefore reduces repeated weight movement while keeping the in-Flash logic simple enough to respect the CMOS area and reliability constraints of NAND devices.

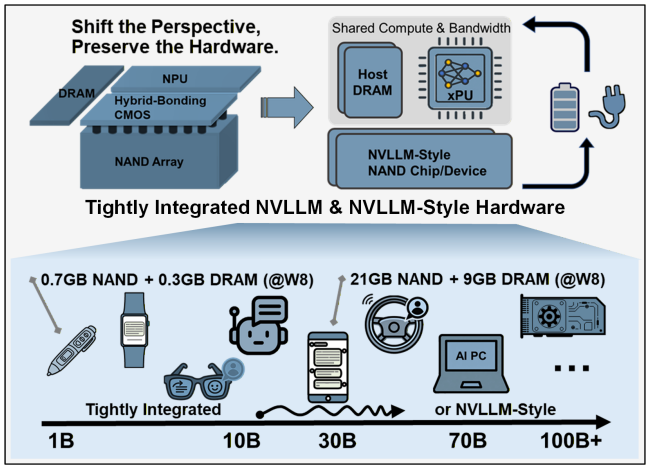

Figure 9: Illustrative deployment scenario of NVLLM in an edge platform. The regular FFN weight-streaming path is served by the 3D NAND-centric execution substrate, while dynamic attention, KV-cache management, and system-level tasks remain on the xPU–DRAM path.

Attention and KV-cache management have different characteristics. Their behavior depends on sequence length, cache layout, sparsity, retrieval, and runtime scheduling policies. NVLLM keeps these components in the xPU–DRAM domain because they benefit from flexible control, random access, and fast adaptation to algorithmic changes. This is not a claim that dynamic operators are outside the scope of future storage-integrated systems. Rather, it is a conservative partition that allows NVLLM to support an evolving workload landscape without forcing every emerging operator into a fixed Flash-side execution model.

In this sense, NVLLM is designed with workload diversity in mind. As retrieval augmentation, sparse attention, speculative decoding, KV-cache optimization, and dynamic routing become more common [6, 18, 21, 22], the system can still preserve a stable in-Flash path for regular model-weight access while leaving dynamic runtime behavior to programmable compute and DRAM. The architecture therefore separates what is structurally stable from what is algorithmically fluid.

A.3 Target Deployment Scenarios

NVLLM is primarily intended for on-device LLM inference on platforms where model capacity exceeds available DRAM capacity. Representative examples include mobile devices, AI PCs, embedded assistants, privacy-sensitive offline applications, and embodied AI. These systems already integrate NAND Flash, DRAM, and CPU/NPU subsystems, making them natural candidates for a 3D NAND-centric execution path.

As illustrated in Fig. 9, a practical advantage of NVLLM is that it preserves the native NAND read/write path and the external storage interface. The Flash subsystem can still serve as conventional storage, while selected inference phases use the NAND-side compute path. This property is important for real edge platforms, where an accelerator architecture must coexist with the operating system, file system, storage controller, and other application workloads.

Although NVLLM is evaluated in the context of LLM inference, the underlying principle is broader. Other emerging AI workloads, such as diffusion transformers (DiTs) [31], vision-language-action (VLA) models [3], and multimodal foundation models [1], may also contain memory-bound phases when deployed under tight edge memory and power budgets. NVLLM does not imply that these workloads should be mapped wholesale into Flash. Instead, it suggests a methodology: when a workload contains large, regular, repeatedly accessed model states alongside dynamic runtime computation, the regular path may be a candidate for near-Flash execution, while the dynamic path remains in programmable memory and compute.

A.4 Outlook: Toward 3D NAND-Native AI Systems

Looking forward, AI-oriented 3D NAND should be optimized not only for storage density, but also for its role in model execution. Capacity, plane-level parallelism, page-buffer organization, read latency, ECC behavior, storage interfaces, and CMOS logic budget should be considered jointly. These factors determine whether high-capacity non-volatile memory can become an effective execution substrate rather than only a passive repository for model weights.

NVLLM is therefore best viewed as an early architectural step rather than a final form of 3D NAND-native AI hardware. Its contribution is to identify a near-term, technology-grounded opportunity: execute the stable, memory-bound portion of inference where the model weights reside, while preserving flexibility for dynamic and rapidly evolving operators. More broadly, NVLLM points to a design direction in which storage, memory, and compute are co-designed around the actual bottlenecks of large AI models, rather than treated as separate layers connected only by data movement.

References

[1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, et al. 2022. Flamingo: a Visual Language Model for Few-Shot Learning. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 23716–23736. doi:10.52202/068431-1723

[2] AMD. 2025. *AMD Ryzen AI Max+ PRO 395.* https://www.amd.com/en/products/processors/laptop/ryzen-pro/ai-max-pro-300-series/amd-ryzen-ai-max-plus-pro-395.html

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, et al. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818 [cs.RO] https://arxiv.org/abs/2307.15818

[4] Karthik Chandrasekar, Christian Weis, Benny Akesson, Norbert Wehn, and Kees Goossens. 2013. Towards variation-aware system-level power estimation of DRAMs: An empirical approach. In *2013 50th ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–8. doi:10.1145/2463209.2488762

[5] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. 2023. Enhancing Chat Language Models by Scaling High-quality Instructional Conversations. *arXiv preprint arXiv:2305.14233* (2023).

[6] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[7] Hua Feng, Debao Wei, Qi Wang, Yongchao Wang, Liyan Qiao, and Zongliang Huo. 2025. Temperature Effects of Program Operation in 3-D Nand Flash Memory: Observations, Analysis, and Solutions. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 9 (Sept. 2025), 3313–3322. doi:10.1109/TCAD.2025.3539982

[8] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. arXiv:2402.02057 [cs.LG] https://arxiv.org/abs/2402.02057

[9] ggml org. 2023. *llama.cpp: Port of Facebook's LLaMA model in C/C++.* https://github.com/ggml-org/llama.cpp

[10] Perttu Hämäläinen, Mikke Tavast, and Anton Kunnari. 2023. Evaluating Large Language Models in Generating Synthetic HCI Research Data: a Case Study. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 433, 19 pages. doi:10.1145/3544548.3580688

[11] Guseul Heo, Sangyeop Lee, Jaehong Cho, Hyunmin Choi, Sanghyeon Lee, Hyungkyu Ham, Gwangsun Kim, Divya Mahajan, and Jongse Park. 2024. Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 722–737.

[12] P. K. Huang, C. Y. Lu, W. H. Wei, Christine Chiu, K. C. Ting, Clark Hu, C.H. Tsai, S. Y. Hou, W. C. Chiou, C. T. Wang, and Douglas Yu. 2021. Wafer Level System Integration of the Fifth Generation CoWoS®-S with High Performance Si Interposer at 2500 Mm2. In *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*. 101–104. doi:10.1109/ECTC32696.2021.00028

[13] HubWeb. 2025. *Apple A19 Pro Chip: Technical Specifications.* https://hubweb.cn/

[14] Zongliang Huo, Weihua Cheng, and Simon Yang. 2022. Unleash Scaling Potential of 3D NAND with Innovative Xtacking® Architecture. In *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. 254–255. doi:10.1109/VLSITechnologyandCir46769.2022.9830285

[15] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. arXiv:2306.06000 [cs.AR] https://arxiv.org/abs/2306.06000

[16] Taesoo Kim, Jiwon Yoon, Seonguk Choi, Haeyeon Kim, Haeseok Suh, Hyunjun An, Jungmin Ahn, Hyunah Park, and Joungho Kim. 2024. Design and Analysis of Twin Tower High Bandwidth Memory (HBM) Architecture for Large Memory Capacity and High Bandwidth System. In *2024 IEEE Electrical Design of Advanced Packaging and Systems (EDAPS)*. 1–3. doi:10.1109/EDAPS64431.2024.10988462

[17] Cheng-Ta Ko, Kuan-Neng Chen, Wei-Chung Lo, Chuan-An Cheng, Wen-Chun Huang, Zhi-Cheng Hsiao, Huan-Chun Fu, and Yu-Hua Chen. 2010. Wafer-Level 3D Integration Using Hybrid Bonding. In *2010 IEEE International 3D Systems Integration Conference (3DIC)*. 1–4. doi:10.1109/3DIC.2010.5751463

[18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[19] Hunjun Lee, Minseop Kim, Dongmoon Min, Joonsung Kim, Jongwon Back, Honam Yoo, Jong-Ho Lee, and Jangwoo Kim. 2022. 3D-FPIM: An Extreme Energy-Efficient DNN Acceleration System Using 3D NAND Flash-Based In-Situ PIM Unit. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, Chicago, IL, USA, 1359–1376. doi:10.1109/MICRO56248.2022.00093

[20] Jaeyong Lee, Hyeunjoo Kim, Sanghun Oh, Myoungjun Chun, Myungsuk Kim, and Jihong Kim. 2025. AiF: Accelerating On-Device LLM Inference Using In-Flash Processing. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 529–543. doi:10.1145/3695053.3731073

[21] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 19274–19286.

[22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9459–9474.

[23] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. arXiv:2401.15077 [cs.LG] https://arxiv.org/abs/2401.15077

[24] Liqiang Lu, Yicheng Jin, Hangrui Bi, Zizhang Luo, Peng Li, Tao Wang, and Yun Liang. 2021. Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 977–991.

[25] Haocong Luo, Yahya Can Tuğrul, F. Nisa Bostancı, Ataberk Olgun, A. Giray Yağlıkçı, , and Onur Mutlu. 2023. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator.

[26] Rino Micheloni, Seiichi Aritome, and Luca Crippa. 2017. Array Architectures for 3-D NAND Flash Memories. *Proc. IEEE* 105, 9 (Sept. 2017), 1634–1649. doi:10.1109/JPROC.2017.2697000

[27] NVIDIA. 2020. *NVIDIA A100 Tensor Core GPU.* https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet.pdf

[28] NVIDIA. 2025. *GeForce RTX 50 Series Laptops.* https://www.nvidia.cn/geforce/laptops/50-series/

[29] OpenAssistant. 2023. *OpenAssistant Conversations Dataset (OASST1).* https://huggingface.co/datasets/OpenAssistant/oasst1/tree/main

[30] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019

[31] William Peebles and Saining Xie. 2023. Scalable Diffusion Models with Transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. 4172–4182. doi:10.1109/ICCV51070.2023.00387

[32] ShareAI Lab. 2023. *ShareGPT-Chinese-English-90k: A Bilingual Chinese-English Human-Machine Dialogue Dataset.* https://huggingface.co/datasets/shareAI/ShareGPT-Chinese-English-90k

[33] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Re, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 31094–31116. https://proceedings.mlr.press/v202/sheng23a.html

[34] Thierry Tambe, Jeff Zhang, Coleman Hooper, Tianyu Jia, Paul N. Whatmough, Joseph Zuckerman, Maico Cassel Dos Santos, Erik Jens Loscalzo, Davide Giri, Kenneth Shepard, Luca Carloni, Alexander Rush, David Brooks, and Gu-Yeon Wei. 2023. 22.9 A 12nm 18.1TFLOPs/W Sparse Transformer Processor with Entropy-Based Early Exit, Mixed-Precision Predication and Fine-Grained Power Management. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*. 342–344. doi:10.1109/ISSCC42615.2023.10067817

[35] TechInsights. 2024. *A Brief View of YMTC Xtacking4.0 Technology: What's New in the Chip?* https://www.techinsights.com/blog/brief-view-ymtc-xtacking40-technology-whats-new-chip

[36] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 97–110. arXiv:2012.09852 [cs] doi:10.1109/HPCA51647.2021.00018

[37] Hanrui Wang, Zhekai Zhang, and Song Han. 2021. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 97–110.

[38] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. 2024. On-Device Language Models: A Comprehensive Review. arXiv:2409.00088 [cs.CL] https://arxiv.org/abs/2409.00088

[39] Zihao Yi, Jiarui Ouyang, Zhe Xu, Yuwen Liu, Tianhao Liao, Haohao Luo, and Ying Shen. 2024. A survey on recent advances in llm-based multi-turn dialogue systems. *Comput. Surveys* (2024).

[40] Dube Belinda Langelihle Yolanda. 2022. Wafer to Wafer Bonding to Increase Memory Density. In *2022 China Semiconductor Technology International Conference (CSTIC)*. 1–4. doi:10.1109/CSTIC55103.2022.9856846

[41] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.

[42] Zhongkai Yu, Shengwen Liang, Tianyun Ma, Yunke Cai, Ziyuan Nan, Di Huang, Xinkai Song, Yifan Hao, Jie Zhang, Tian Zhi, Yongwei Zhao, Zidong Du, Xing Hu, Qi Guo, and Tianshi Chen. 2024. Cambricon-LLM: A Chiplet-Based Hybrid Architecture for On-Device Inference of 70B LLM. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1474–1488. doi:10.1109/MICRO61859.2024.00108

[43] Shulin Zeng, Jun Liu, Guohao Dai, Xinhao Yang, Tianyu Fu, Hongyi Wang, Wenheng Ma, Hanbo Sun, Shiyao Li, Zixiao Huang, Yadong Dai, Jintao Li, Zehao Wang, Ruoyu Zhang, Kairui Wen, Xuefei Ning, and Yu Wang. 2024. FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs. arXiv:2401.03868 [cs] doi:10.48550/arXiv.2401.03868

[44] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. 2025. A review on edge large language models: Design, execution, and applications. *Comput. Surveys* 57, 8 (2025), 1–35.

[45] Zhe Zhou, Junlin Liu, Zhenyu Gu, and Guangyu Sun. 2022. Energon: Toward efficient acceleration of transformers using dynamic sparse attention. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 1 (2022), 136–149.